

FACULTY OF INFORMATION AND COMMUNICATION TECHNOLOGY

Declaration

Plagiarism is defined as “the unacknowledged use, as one’s own work, of work of another person, whether or not such work has been published” (Regulations Governing Conduct at Examinations, 1997, Regulation 1 (viii), University of Malta).

I / We*, the undersigned, declare that the [assignment / Assigned Practical Task report / Final Year Project report] submitted is my / our* work, except where acknowledged and referenced.

I / We* understand that the penalties for making a false declaration may include, but are not limited to, loss of marks; cancellation of examination results; enforced suspension of studies; or expulsion from the degree programme.

Work submitted without this signed declaration will not be corrected, and will be given zero marks.

* Delete as appropriate.

(N.B. If the assignment is meant to be submitted anonymously, please sign this form and submit it to the Departmental Officer separately from the assignment).

Student Name

Signature

Student Name

Signature

Student Name

Signature

Student Name

Signature

Course Code

Title of work submitted

Date

CNN Feature Visualisation and DeepDream

ARI5118 - Study Notes

David Farrugia (0015004H)
david.farrugia.22@um.edu.mt
M.Sc. in Artificial Intelligence
University of Malta
Msida, Malta

“What is a CNN actually looking at?”

1 The Why and How

Convolutional Neural Networks (CNNs) **achieve high performance** in visual tasks, yet they are **often described as being “black boxes”**. While they can classify images with great accuracy, it is not immediately clear what internal features they rely on or why a particular decision is made. This lack of transparency is an issue especially in domains where interpretability is important, such as medical imaging or autonomous systems.

Feature visualisation techniques have been widely studied, with comprehensive surveys such as [5] (Nguyen et al.) providing an overview of different approaches and their role in improving interpretability. Among these approaches, optimisation-based methods such as activation maximisation and DeepDream are very important, as they address this problem by shifting the perspective from training the model to probing the model. Instead of asking “How do we improve predictions?”, we ask:

“What kind of input would make this neuron respond the most?”

An good way to understand this is to picture each neuron as having a “preference”. Just as a human might prefer certain visual patterns (e.g., faces, colours, games, screen brightness, the laptops/machines we use day-to-day... etc.), **a neuron in a CNN responds strongly to specific patterns in the input**. Feature visualisation attempts to reverse-engineer these preferences.

This is achieved by treating the **input image as a variable that can be optimised**. Rather than updating the **network weights (as in training)**, the **model parameters are kept fixed**, and the **input image is iteratively modified** to produce **stronger activations**. Over multiple iterations, this reveals the **patterns that the network has learned internally**.

A key distinction in feature visualisation is between **analysing real images** and **generating new ones**. Methods such as **saliency maps** work on existing images to **highlight important regions**, while **activation maximisation creates entirely new images** that represent what a neuron has learned. This distinction is important because it highlights two **complementary perspectives: explaining model decisions and uncovering internal representations**. As discussed in [1] (Olah et al.), these approaches show that CNNs **do not detect objects directly**, but instead learn from **simpler visual features across layers**.

2 Essential Algorithms

2.1 Activation Maximisation

Activation maximisation is an optimisation-based method used to generate an input image that maximises the activation of a specific neuron or feature map within a trained CNN. The network parameters remain fixed, and the input image is iteratively updated.

The objective is defined as:

$$x^* = \arg \max_x a_i(x) \quad (1)$$

Since this optimisation cannot be solved analytically, it is approximated using gradient ascent:

$$x_{t+1} = x_t + \eta \frac{\partial a_i(x)}{\partial x} \quad (2)$$

where x_t is the input at iteration t , x_{t+1} is the updated input after one optimisation step and η is the step size (how big change is). As shown in [1] (Olah et al.), this process gradually modifies the input image to reveal the preferred stimulus of the neuron.

In simple terms, take the current image and slightly change it in the direction that increases activation. Other gradient-based methods, such as guided backpropagation improve the clarity of visualisations by modifying gradient flow.

However, without constraints, the optimisation often produces noisy and high-frequency patterns rather than meaningful visual structures.

2.2 Regularisation

To produce interpretable images, regularisation terms are introduced to guide the optimisation towards natural-looking patterns. The objective becomes:

$$x_{t+1} = x_t + \eta \left(\frac{\partial a_i(x)}{\partial x} - \lambda \frac{\partial R(x)}{\partial x} \right) \quad (3)$$

Common regularisation techniques include: **L2 decay** to penalise large pixel values and **Gaussian blur** to suppress high-frequency noise. As discussed in [4] (Mahendran & Vedaldi), these constraints help prevent the optimisation from exploiting unrealistic pixel-level artefacts and instead produce more meaningful visual representations.

2.3 DeepDream

DeepDream extends activation maximisation by amplifying patterns already present in a real image rather than generating images from noise.

Instead of maximising a single neuron, DeepDream maximises the activations of an entire layer:

$$x_{t+1} = x_t + \eta \frac{\partial}{\partial x} \left(\sum_{i \in \text{layer}} a_i(x) \right) \quad (4)$$

This causes the network to reinforce and exaggerate features it detects, leading to the “hallucinated” patterns described in [2] (Mordvintsev et al.). Lower layers enhance textures such as edges, while deeper layers amplify complex structures such as object parts.

3 Worked Numerical Example

To understand how gradient ascent works, let's start with a very simple example.

Consider a neuron with the following activation function:

$$a(x) = 2x \tag{5}$$

This means that as the input x increases, the activation also increases. In other words, **larger values of x produce stronger neuron responses.**

The rate at which the activation changes with respect to the input is given by the gradient:

$$\frac{\partial a}{\partial x} = 2 \tag{6}$$

This tells us that for every small increase in x , the activation increases by a constant amount. Since the gradient is positive, increasing x will always increase the activation.

To apply gradient ascent, which updates the input in the direction that increases the activation. Starting from an initial value $x_0 = 1$ and using a step size (learning rate) $\eta = 0.1$, we update x iteratively:

$$x_1 = 1 + 0.1 \cdot 2 = 1.2 \tag{7}$$

$$x_2 = 1.2 + 0.1 \cdot 2 = 1.4 \tag{8}$$

$$x_3 = 1.4 + 0.1 \cdot 2 = 1.6 \tag{9}$$

At each step, the input value increases because the gradient consistently points in the direction of higher activation. This simple example shows how the input is gradually adjusted to maximise the neuron's response.

In Convolutional Neural Networks (CNNs), the same principle applies. However, instead of a single value x , the input is an image composed of many pixels. The gradient is therefore computed with respect to all pixel values, indicating how each pixel should be adjusted to increase the activation. As noted in [1] (Olah et al.), this allows the optimisation process to reveal patterns that maximise neuron responses.

4 Feature Hierarchy in CNNs

CNNs learn hierarchical representations of visual data. Early layers detect low-level features such as edges and textures, while deeper layers capture more complex patterns such as object parts and full objects.

This hierarchical transformation can be expressed as:

$$h^{(l)} = \sigma \left(W^{(l)} * h^{(l-1)} + b^{(l)} \right) \tag{10}$$

As shown in [1] (Olah et al.) and [6] (Zeiler & Fergus), feature visualisation reveals that:

- Early layers respond to simple patterns (edges, colours)
- Middle layers detect textures and shapes
- Deep layers capture semantic concepts

This aligns with [6] (Zeiler & Fergus), which highlights how different layers capture increasingly complex features, helping to explain why DeepDream produces simple textures in shallow layers and more complex, object-like patterns in deeper layers.

5 Real-World Applications

Feature visualisation techniques are used in many areas as showing in **Figure. 1**, especially for understanding and improving deep learning models. One major use case is **model debugging**, where these methods help show what features a network has learned and whether it is relying on incorrect patterns or biases [3] (Simonyan et al.). By visualising neuron activations, researchers can better understand how the model behaves and identify possible issues.

Furthermore, in medical imaging, feature visualisation helps explain model decisions by showing which patterns influence a diagnosis. Methods such as Grad-CAM [7] (Selvaraju et al.) highlight the regions in the input that most strongly contribute to the model's prediction. This is especially important in high-risk applications, where understanding the reasoning behind predictions helps build trust and allows results to be validated.

Additionally, feature visualisation is also important for studying **adversarial robustness**. This simply means **how models can be affected by small, unnatural changes in input data**. This helps researchers understand how adversarial examples work and how to make models more reliable.

Lastly, these techniques are also widely used in **art and creative use cases**. For example, DeepDream generates unique "strange" images by enhancing patterns learned by a network, showing the creative capabilities of deep learning systems [2] (Mordvintsev et al.).

Real-World Applications of Feature Visualisation in CNNs

Feature visualisation helps us understand what a CNN has learned, improve model trust, and enable creative applications.

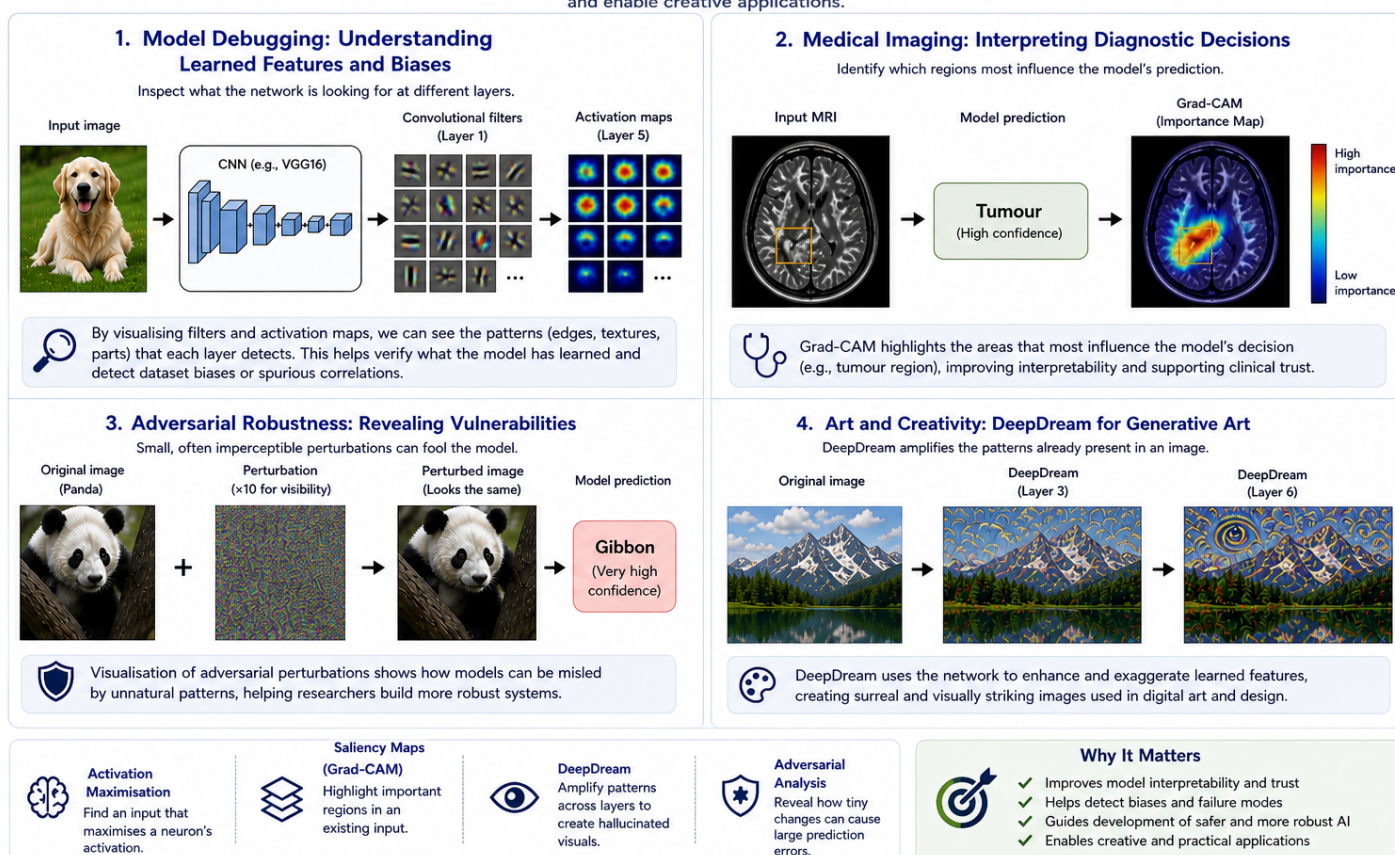


Figure 1: Real-world applications of feature visualisation across different domains

6 Quick Reference:

Key Concepts:

- Activation maximisation: Optimising input to maximise neuron activation
- Gradient ascent: Iteratively updating input in direction of gradient
- DeepDream: Amplifying patterns in an existing image
- Feature hierarchy: Layers learn progressively complex representations

Key Equations:

$$x_{t+1} = x_t + \eta \frac{\partial a_i(x)}{\partial x} \quad (11)$$

$$x_{t+1} = x_t + \eta \left(\frac{\partial a_i(x)}{\partial x} - \lambda \frac{\partial R(x)}{\partial x} \right) \quad (12)$$

Common Misconceptions:

- CNNs do not detect objects directly, but patterns
- Activation maximisation does not reconstruct real images
- DeepDream does not “see” objects, it amplifies learned features

Blogs/Articles:

[1]C. Olah, A. Mordvintsev, and L. Schubert, "Feature Visualization," *Distill*, vol. 2, no. 11, Nov. 2017, doi:<https://doi.org/10.23915/distill.00007>.

Summary: This article explains feature visualisation as a method to understand what neural networks detect by generating inputs that strongly activate parts of the network. It focuses on optimisation-based methods (like activation maximisation) and discusses practical challenges, such as producing noisy or unrealistic images and the need for regularisation. The paper also emphasises that feature visualisation helps reveal how networks build understanding across layers.

Relevance: Discusses activation maximisation, gradient-based visualisation, Feature hierarchy across layers.

[2]Mordvintsev et.al.. "Inceptionism: Going Deeper into Neural Networks," *research.google*, <https://research.google/blog/inceptionism-going-deeper-into-neural-networks/>

Summary: This article introduces DeepDream as a technique that modifies images so a CNN enhances patterns it already detects, often producing surreal results. The network “over-interprets” visual patterns based on what it has learned (e.g., seeing animals in clouds). The process effectively runs the network in reverse by adjusting the input image to increase activations.

Relevance: Key source for understanding DeepDream amplification and how CNNs visualise and hallucinate learned patterns.

7 Key Papers:

[3]K. Simonyan, A. Vedaldi, and A. Zisserman, "Deep Inside Convolutional Networks: Visualising Image Classification Models and Saliency Maps," arXiv.org, 2013. <https://arxiv.org/abs/1312.6034>

Summary: This paper introduced activation maximisation as a method to visualise CNN representations. It proposed generating “class model” images by optimising input pixels to maximise class scores. The work highlighted how gradients with respect to input can reveal what features a network relies on. It also introduced saliency maps, providing insight into pixel importance.

Relevance: Foundation of gradient-based feature visualisation.

[4]A. Mahendran and A. Vedaldi, "Visualizing Deep Convolutional Neural Networks Using Natural Pre-images," International Journal of Computer Vision, vol. 120, no. 3, pp. 233{255, May 2016, doi: <https://arxiv.org/abs/1512.02017>.

Summary: This paper explores how to reconstruct input images from CNN feature representations. It introduces optimisation-based inversion techniques combined with regularisation to produce natural-looking reconstructions. The study shows how different layers retain varying levels of information, demonstrating that early layers preserve fine details while deeper layers retain more abstract, semantic content. The authors also highlight the importance of priors (regularisation) in ensuring interpretable outputs.

Relevance: Provides mathematical grounding for regularisation in activation maximisation and connects directly to feature hierarchy.

[5]A. Nguyen, J. Yosinski, and J. Clune, "Understanding Neural Networks via Feature Visualization: A survey," arXiv.org, 2019. <https://arxiv.org/abs/1904.08939>

Summary: This survey paper provides a comprehensive overview of feature visualisation techniques, including activation maximisation, optimisation-based methods, and attribution methods. It categorises different approaches and discusses their strengths and limitations. The paper also introduces the idea of synthesising “preferred stimuli” for neurons and explores how these methods contribute to interpretability in deep learning systems.

Relevance: Clarifies the theoretical foundations behind optimisation-based visualisation techniques and how they relate to interpretability.

[6]M. D. Zeiler and R. Fergus, "Visualizing and Understanding Convolutional Networks," arXiv.org, 2013. <https://arxiv.org/abs/1311.2901>

Summary: This paper introduces a deconvolutional network approach to visualise intermediate feature activations in CNNs. By projecting feature maps back into input space, the authors show what each layer responds to. The work demonstrates the hierarchical nature of CNN representations and identifies issues such as dead filters and poor training configurations. It was instrumental in improving CNN architectures by making their behaviour interpretable.

Relevance: Provides insight into how internal CNN representations can be visualised and interpreted, helping to understand what features the network learns at different stages.

[7]R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, "Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization," International Journal of Computer Vision, vol. 128, no. 2, pp. 336{359, Feb. 2020, doi: <https://arxiv.org/abs/1610.02391>.

Summary: Grad-CAM introduces a method to produce class-discriminative localisation maps by using gradients flowing into the final convolutional layer. It highlights important regions in an image that influence a prediction. Unlike activation maximisation, which generates

synthetic inputs, Grad-CAM operates on real images and provides spatial explanations.

Relevance: Useful contrast: input-space vs activation-space interpretability.